



Error Checking Algorithms

Hamming Code
Cyclic Redundancy Check
BCH

Aush Shukla
200907282

Aaryan Alexander Martis
200907256

Sandali Singh
200907044



Objective

This project provides a comprehensive overview of essential error detection and correction techniques and their practical implementations in Python. By utilizing an interactive approach, the report aims to enhance the reader's understanding of how these techniques can be applied in real-world scenarios to ensure data accuracy and integrity during transmission over unreliable channels.



Error control and check

Error control and checking are crucial in communication systems to ensure the accuracy and reliability of transmitted data. In any communication process, especially in digital data transmission, errors can occur due to various factors such as noise, interference, distortion, or hardware/software malfunctions. Implementing error control and checking mechanisms helps in the detection, correction, and prevention of these errors, thus ensuring the integrity of the data being transmitted.

We can make use of various standard algorithms for this, some of which are:

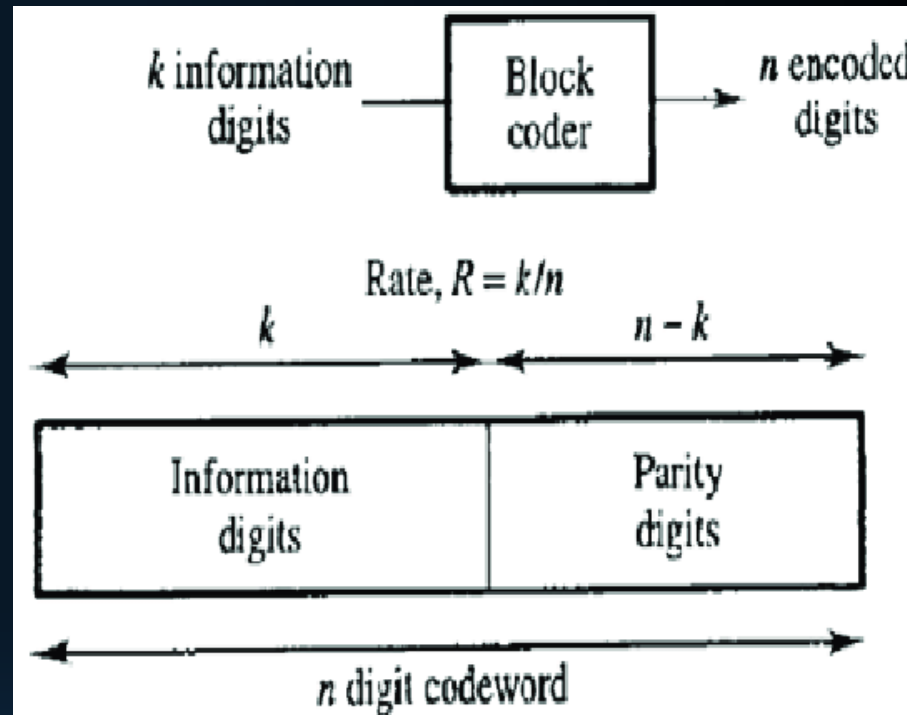
- Linear Block Code
- BCH



Hamming Code

The simulation process for Hamming Code involves selecting the code, encoding the message, simulating a noisy channel, decoding the message, detecting and correcting errors, and calculating the Bit Error Rate (BER) to evaluate error correction accuracy.

The Hamming Code is a linear block code designed for error detection and correction purposes, and the Hamming(7,4) encoder produces a 7-bit encoded codeword from a 4-bit message



Hamming Code

The parity-check matrix is used to detect and correct errors in the received codeword. Finally, the BER is calculated by comparing the original message with the decoded one.

It is a simple error control coding technique used for error *detection* and *correction*.

- Information data is partitioned into blocks of length k pieces for example Information word.
- Every information word is then coded into a block of length n bits called a codeword. Here, $n > k$
- Also, $n = k + r$, where ' r ' denotes the parity bits or check bits added to every information word.
- Vector documentation is utilized for the Data word and Codeword: message $m = (m_1, m_2, m_n)$, Codeword $c = (c_1, c_2, c_n)$.

Result

```
Hamming(7,4) Code Simulation
Enter a 4-bit message (e.g., 4 binary digits): 1101
Enter the noise probability (e.g., 0.1 for 10% noise): 0.5

Simulation Results:
Original Message: [1 1 0 1]
Encoded Codeword: [1 1 0 1 1 0 0]
Decoded Message: [1 0 0 1]
PS C:\Users\ayush>
```

(with some error)

```
Hamming(7,4) Code Simulation
Enter a 4-bit message (e.g., 4 binary digits): 1101
Enter the noise probability (e.g., 0.1 for 10% noise): 0

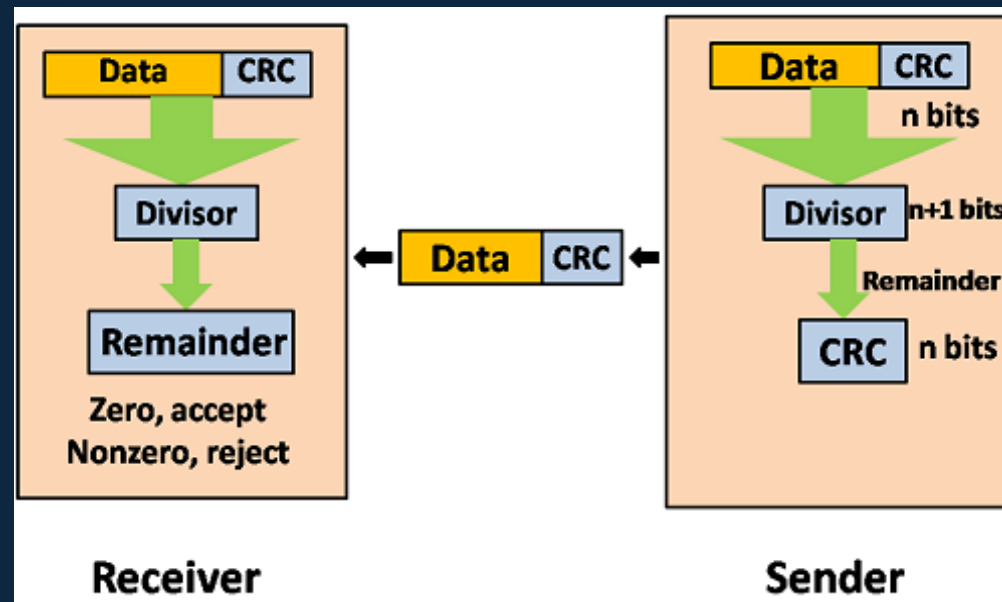
Simulation Results:
Original Message: [1 1 0 1]
Encoded Codeword: [1 1 0 1 1 0 0]
Decoded Message: [1 1 0 1]
PS C:\Users\ayush>
```

(without any error)



Cyclic Redundancy Check (CRC)

CRC, which stands for Cyclic Redundancy Check, is an error-detection technique commonly used in digital networks and storage devices to detect accidental changes to raw data. It works by generating a checksum, which is a fixed-size bit string, based on the data that is being transmitted. The checksum is appended to the data, and the recipient can use it to verify the integrity of the received data.



Cyclic Redundancy Check (CRC)

The key advantage of CRC is its effectiveness in detecting a variety of errors, such as single-bit errors and most common multiple-bit errors. However, it is important to note that CRC does not provide error correction capabilities; it can only detect errors. Additionally, the efficiency of CRC depends on the chosen generator polynomial, as certain polynomials may be more suitable for specific types of data and applications.

CRC is widely used in various communication protocols, such as Ethernet, USB, and SATA, to ensure the accuracy and reliability of data transmission. Its simplicity, efficiency, and versatility have made it one of the most popular error-detection techniques in digital communication systems.

Result

1. Encode data
2. Check data at the receiver side
3. Exit

Enter your choice: 1

Enter data to encode: 100100

Enter key for encoding: 1101

Remainder: 001

Data: 100100001



BCH

In coding theory, the Bose–Chaudhuri–Hocquenghem codes (BCH codes) form a class of cyclic error-correcting codes that are constructed using polynomials over a finite field (also called a Galois field).

One of the key features of BCH codes is that during code design, there is a precise control over the number of symbol errors correctable by the code. In particular, it is possible to design binary BCH codes that can correct multiple bit errors.

Another advantage of BCH codes is the ease with which they can be decoded, namely, via an algebraic method known as syndrome decoding. This simplifies the design of the decoder for these codes, using small low-power electronic hardware.

$$\begin{bmatrix} \alpha^{ck_1} & \alpha^{ck_2} & \dots & \alpha^{ck_{d-1}} \\ \alpha^{(c+1)k_1} & \alpha^{(c+1)k_2} & \dots & \alpha^{(c+1)k_{d-1}} \\ \vdots & \vdots & & \vdots \\ \alpha^{(c+d-2)k_1} & \alpha^{(c+d-2)k_2} & \dots & \alpha^{(c+d-2)k_{d-1}} \end{bmatrix} \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_{d-1} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}.$$

BCH

BCH codes are used in applications such as satellite communications, compact disc players, DVDs, disk drives, USB flash drives, solid-state drives, quantum-resistant cryptography and two-dimensional bar codes.

Result

```
● PS C:\Users\ayush> python -u "d:\Study\MIT manipal ece\MIT 4th year\7th semester\ECC\fisac\parity_check.py"
Enter the Original Message in the form of bits: 1010101
Original Message (BCH): [1 0 1 0 1 0 1]
Encoded Codeword (BCH): [1 0 1 0 1 0 1]
Noisy Codeword (BCH): [1 0 1 0 1 0 1]
Decoded Message (BCH): [1 0 1 0 1 0 1]
Bit Error Rate (BER) for BCH Code: 0.15317
○ PS C:\Users\ayush> █
```

Thank You

